OBJECT ORIENTED PROGRAMMING

S. VINILA JINNY

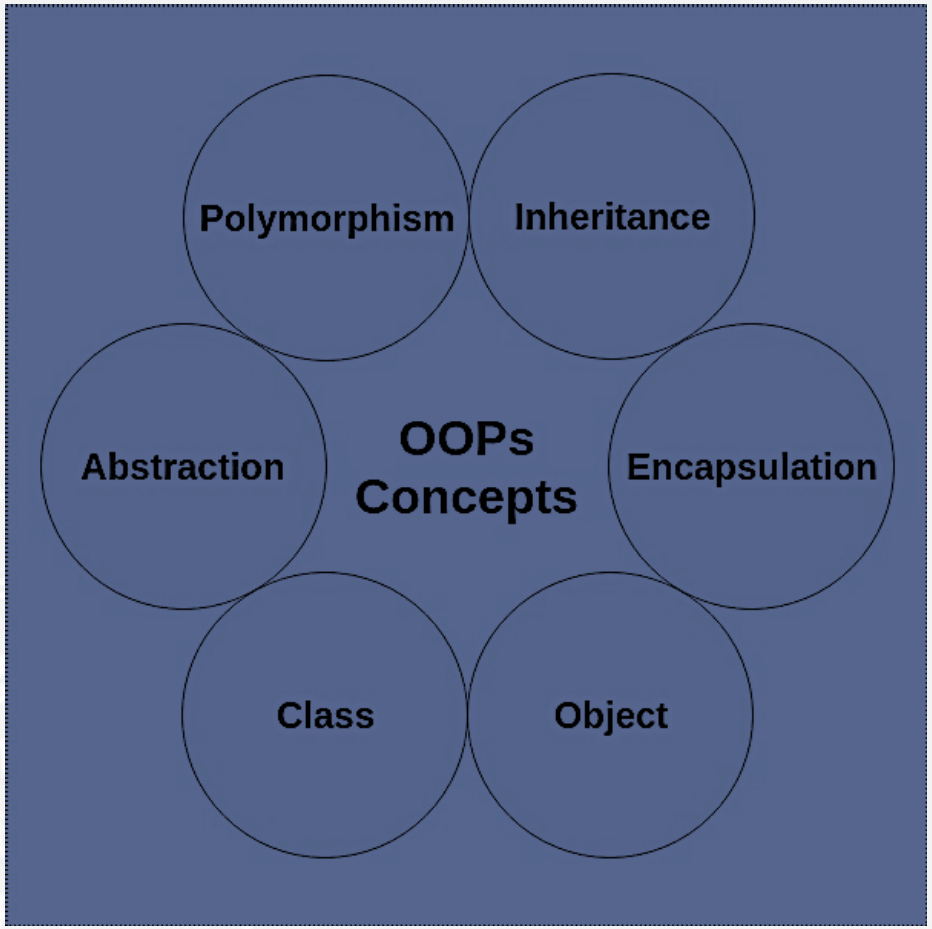
OOP

- OOP stands for Object-Oriented Programming.
- object-oriented programming is about creating objects that contain both data and functions.
- While, Procedural programming is about writing procedures or functions that perform operations on the data

ADVANTAGES OF OOP

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

OOP CONCEPTS



CLASS

- A class in C++ is the building block that leads to Object-Oriented programming.
- It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- A C++ class is like a blueprint for an object.
- Example: class car – can have different types of car under this.
- properties : 4 wheels, Speed Limit, mileage

OBJECT

- An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- An Object is an entity with some characteristics and behaviour.

Features of OOP

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Encapsulation

- “encapsulate,” means to enclose something
- **Encapsulation** is defined as wrapping up of data and information under a single unit.
- *Encapsulation* is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.
- In an object-oriented language, code and data may be combined in such a way that a self-contained "black box" is created.
- When code and data are linked together in this fashion, an *object* is created. In other words, an object is the device that supports encapsulation.

Abstraction

- Data abstraction is one of the most essential and important feature of object oriented programming in C++.
- Abstraction means displaying only essential information and hiding the details.
- Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

INHERITANCE

- The capability of a class to derive properties and characteristics from another class is called **Inheritance**.
- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class. **Super Class:** The class whose properties are inherited by sub class is called Base Class or Super class.

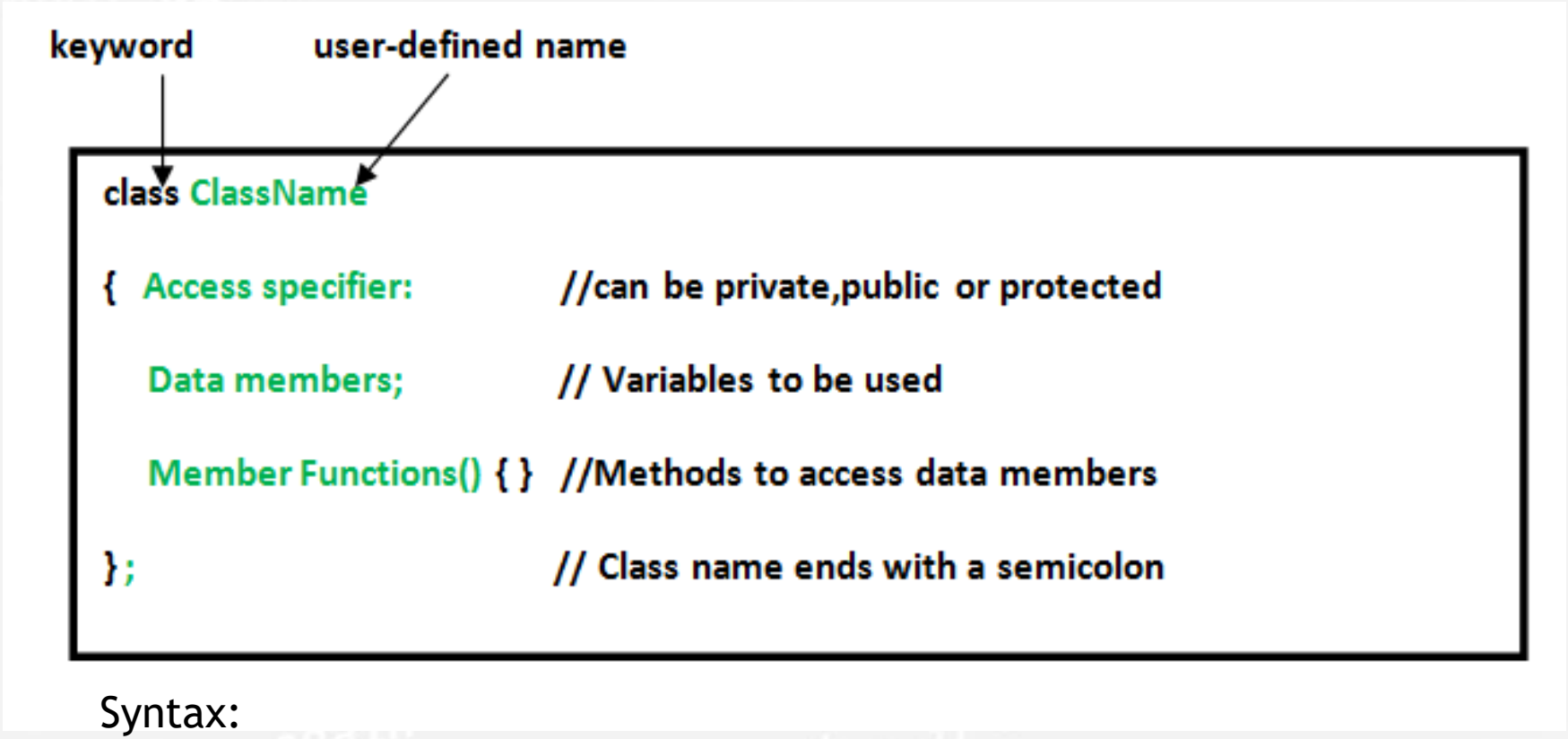
Polymorphism

- In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form
- The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms.
- It is a greek word.

Sample c++ program

```
#include <iostream>
int main()
{
int i;
cout << "This is output.\n"; // this is a single line comment
/* you can still use C style comments */
// input a number using >>
cout << "Enter a number: ";
cin >> i;
// now, output a number using <<
cout << i << " squared is " << i*i << "\n";
return 0;
}
```

Defining Class and Declaring object



Syntax:

ClassName ObjectName;

Class and object

```
class person
{
        char name[20];
        int id;
public:
        void getdetails(){}
};

int main()
{
person p1; // p1 is a object
}
```

Accessing data members and member functions

- The data members and member functions of class can be accessed using the dot('.') operator with the object.

```
#include <iostream>
using namespace std;
class Student {
   public:
      int id;//data member (also instance variable)
      string name;//data member(also instance variable)
};
int main() {
    Student s1; //creating an object of Student
    s1.id = 201;
    s1.name = "Sonoo Jaiswal";
    cout<<s1.id<<endl;
    cout<<s1.name<<endl;
    return 0;
}
```

Member functions

- There are 2 ways to define a member function:
- Inside class definition
- Outside class definition
- To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.

Member function inside class

```
#include <iostream>
class Student {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     void insert(int i, string n)
      {
         id = i;
         name = n;
      }
     void display()
      {
         cout<<id<<"  "<<name<<endl;
      }
};
```

```
int main() {
    Student s1; //creating an object of Student
    Student s2; //creating an object of Student
    s1.insert(201, "Sonoo");
    s2.insert(202, "Nakul");
    s1.display();
    s2.display();
    return 0;
}
```

Member function outside class

```
#include <iostream>
class Student {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     void insert(int i, string n);
void display()
     {
        cout<<id<<"  "<<name<<endl;
     }
};
```

```
void Student :: insert(int i, string n)
     {
        id = i;
        name = n;
     }
int main() {
   Student s1; //creating an object of Student
   Student s2; //creating an object of Student
   s1.insert(201, "Sonoo");
   s2.insert(202, "Nakul");
   s1.display();
   s2.display();
   return 0;
}
```

Inline Concept Visited

- All the member functions defined inside the class definition are by default **inline**,
- But we can also make any non-class function inline by using keyword inline with them.
- Inline functions are actual functions, which are copied everywhere during compilation, like pre-processor macro, so the overhead of function calling is reduced.

Constructor

- Constructors are special class members which are called by the compiler every time an object of that class is instantiated. Constructors have the same name as the class and may be defined inside or outside the class definition. There are 3 types of constructors:
- Default constructors
- Parameterized constructors
- Copy constructors

Default Constructor

- A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

```
#include <iostream>
class Employee
 {
   public:
        Employee()
        {
            cout<<"Default Constructor Invoked"<<endl;
        }
};
int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2;
    return 0;
}
```

Output:

```
Default Constructor Invoked
Default Constructor Invoked
```

Parametrized Constructor

- **A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.**

```
#include <iostream>
class Employee {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     float salary;
     Employee(int i, string n, float s)
      {
          id = i;
          name = n;
          salary = s;
      }
     void display()
      {
          cout<<id<<"  "<<name<<"  "<<salary<<endl;
      }
};
```

```
int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000);
//creating an object of Employee
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

Output:

```
101  Sonoo  890000
102  Nakul  59000
```